Accelerating FFT Applications on the CV32A6 RISC-V Processor - *Team Sonic*

Martin ROUXEL
martin.rouxel@imt-atlantique.net
IMT Atlantique
Plouzané, France

Thanh-Thang VO
thang.vo@imt-atlantique.net
IMT Atlantique
Plouzané, France

***Abstract*—This paper presents the solution developed at IMT Atlantique for the RISC-V student contest based on the CV32A6 soft-core processor. The objective of the contest is to accelerate an FFT workload while preserving the correctness of the reference applications and the compatibility of the processor with the RV32IM_Zicsr software environment. The baseline FFT implementation manipulates complex values with separate 32-bit real and imaginary components, which leads to high register pressure, memory traffic, and instruction count. Our first optimization is therefore both a numerical and data-layout transformation: the FFT data are converted to a packed fixed-point Q15 representation, where one 32-bit register stores one complete complex value with 16-bit real and imaginary components. On top of this representation, we implement a CV-X-IF coprocessor that provides packed complex arithmetic instructions and stage-aware butterfly instructions for the dominant FFT-512 kernels. The final accelerator combines fused fixed-point scaling, complex multiplication, radix-2 recombination, and a stateful radix-4 butterfly protocol. Simulation and FPGA measurements show a substantial reduction in FFT instruction and cycle counts while preserving functional validation of FFT, CoreMark, and MNIST.**

I. Introduction

The National RISC-V Student Contest is centered on the modification and evaluation of an open-source RISC-V processor. The platform used in the contest is the CV32A6, a 32-bit variant of the CVA6 processor family. The goal of the contest is not only to accelerate one target workload, but also to demonstrate that the proposed architectural modification remains compatible with the reference software flow, the processor programming model, and the FPGA implementation constraints.

This article first presents the challenge context and the execution platform, then explains the baseline workload, introduces the proposed optimizations, and finally reports correctness, performance, timing, and FPGA resource results.

In the current edition, the target application is a 512-point FFT. FFT acceleration is a representative problem for embedded processor customization because the algorithm combines regular arithmetic kernels with strict implementation constraints. The transform is dominated by repeated complex operations, twiddle multiplications, fixed-point normalizations, and butterfly recombinations. A useful acceleration strategy must therefore reduce both arithmetic cost and instruction overhead, while preserving the expected output of the reference application.

Our work proposes a tightly coupled FFT coprocessor connected to CV32A6 through the CV-X-IF interface. The processor still controls the program, loop structure, memory accesses, and FFT stage traversal. The coprocessor only replaces the hot arithmetic kernels with custom instructions. This preserves the software structure of the reference implementation and avoids turning the solution into a standalone FFT engine.

A. Hardware Platform

The target hardware platform is the Digilent Zybo Z7-20 FPGA board. The complete CV32A6-based system is synthesized, placed, routed, and programmed on this board. The final design includes the original processor system and the added CV-X-IF coprocessor. The evaluation therefore includes both simulation results and measurements obtained on the physical FPGA board.

Fig. 1: Target FPGA development board used for validation.

B. Software and Tool Flow

The development flow uses the standard software and hardware tools provided for the CV32A6 contest environment. Questa simulation is used to validate execution and collect simulation cycle counts. Vivado/Vitis is used to synthesize, place, route, and program the FPGA design. Docker is used to simplify access to the RISC-V toolchain and the software build environment.

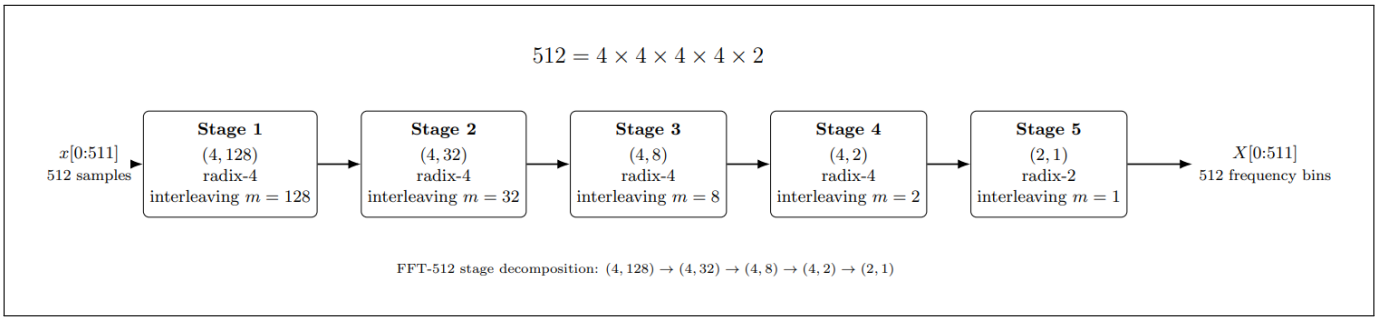

Fig. 2: Stage decomposition of the target FFT-512 workload.

The proposed modifications are evaluated with three software applications. The FFT application is the target workload. CoreMark and MNIST are used as non-regression tests.

C. FFT Context

A Fast Fourier Transform converts a sequence from the time domain to the frequency domain. The contest workload is based on a 512-point KISS FFT. In the baseline software, complex values are manipulated as separate 32-bit real and imaginary components. This representation is simple and portable, but it is not well matched to the integer datapath customization opportunities offered by CV32A6 and CV-X-IF.

The targeted FFT size is 512. Its factorization is

$$\{4, 128,\ 4, 32,\ 4, 8,\ 4, 2,\ 2, 1\} \quad \text{(see fig. 2,3)},$$

which means that the transform contains four radix-4 stages followed by one radix-2 stage. This structure directly guided the accelerator design: the radix-4 stages dominate the arithmetic cost and deserve the most specialized support, while the radix-2 tail benefits from a smaller fused instruction sequence.

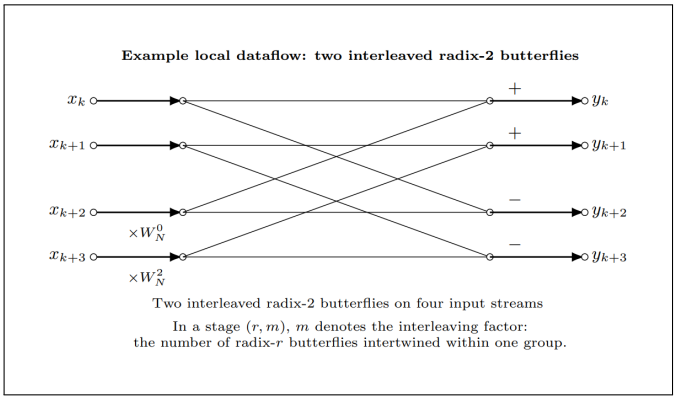

Fig. 3: Example of interleaved butteflies

II. FFT Baseline

This section describes how the reference FFT workload is computed before introducing any hardware-specific optimization. The objective is to identify the arithmetic structure of the application and the sources of execution cost on a scalar 32-bit RISC-V processor.

A. Complex Arithmetic in the Reference Implementation

The reference implementation represents each complex value as two scalar components: a real part and an imaginary part. A complex number is therefore manipulated as

$$x = x_r + jx_i,$$

where x_r and x_i are stored and processed separately.

The most common operation in the FFT is multiplication by a twiddle factor. Given an input value

$$x = x_r + jx_i$$

and a twiddle factor

$$W = W_r + jW_i,$$

the product is computed as

$$xW = (x_rW_r - x_iW_i) + j(x_rW_i + x_iW_r).$$

On a scalar processor, this single complex multiplication requires four integer multiplications, two additions or subtractions, and several temporary values. Listing 1 shows the typical scalar structure of this operation.

Listing 1: Baseline complex multiplication using separate real and imaginary scalar components.

```
// ar = real(a), ai = imag(a), br = real(b), bi = imag(b)
t0 = ar * br;
t1 = ai * bi;
t2 = ar * bi;
t3 = ai * br;
real = t0 - t1;
imag = t2 + t3;
```

This decomposition is portable and easy to express in C, but it creates a large number of scalar instructions for each complex operation.

B. Radix-2 Butterfly

A radix-2 butterfly combines two complex inputs. Let x_0 and x_1 be the two inputs, and let W be the twiddle factor applied to the second input. The butterfly first computes

$$t = x_1W, \quad \text{then produces} \quad y_0 = x_0 + t, \quad y_1 = x_0 - t.$$

Therefore, even a radix-2 butterfly requires one complex multiplication and two complex additions or subtractions.

With separate real and imaginary components, this expands into several scalar arithmetic instructions and multiple temporary values.

C. Radix-4 Butterfly

A radix-4 butterfly combines four complex inputs x_0, x_1, x_2, and x_3. In the general case, three of these inputs are multiplied by twiddle factors:

$$t_0 = x_0, \quad t_1 = x_1 W_1, \quad t_2 = x_2 W_2, \quad t_3 = x_3 W_3.$$

The butterfly then forms intermediate sums and differences:

$$a_0 = t_0 + t_2, \qquad a_1 = t_0 - t_2,$$
$$b_0 = t_1 + t_3, \qquad b_1 = t_1 - t_3.$$

For a forward FFT, the outputs can be expressed as

$$y_0 = a_0 + b_0, \quad y_2 = a_0 - b_0,$$
$$y_1 = a_1 - jb_1, \quad y_3 = a_1 + jb_1.$$

The terms involving j correspond to quarter-turn rotations in the complex plane. For a complex value $z = z_r + jz_i$,

$$jz = -z_i + jz_r \quad \text{and} \quad -jz = z_i - jz_r.$$

A radix-4 butterfly is therefore significantly more expensive than a radix-2 butterfly. It contains three complex multiplications by twiddle factors, several complex additions and subtractions, and two rotations by j or $-j$. Since the 512-point workload contains four radix-4 stages, this operation dominates the execution time of the baseline FFT.

D. Baseline Cost on CV32A6

The baseline implementation maps these complex operations onto the standard RV32IM integer datapath. Since each complex value is represented by separate real and imaginary scalar components, the processor must execute several instructions for each mathematical operation. The main sources of overhead are:

- multiple scalar multiplications for each complex multiplication;
- separate additions and subtractions for real and imaginary components;
- temporary values created during butterfly recombination;
- increased register pressure due to separate real and imaginary variables;
- additional memory traffic when intermediate complex values cannot remain in registers.

As a result, the FFT workload is not limited only by multiplication latency. It is also limited by instruction count, data movement, register pressure, and repeated scalar recombination patterns. These observations motivate the optimizations introduced in the next section.

III. Proposed Optimizations

This section presents the three main ideas used to accelerate the FFT application: a packed fixed-point data representation, packed complex custom instructions, and butterfly-level specialization.

A. Packed Q15 Complex Representation

In the baseline implementation, the real and imaginary parts of a complex value are stored and processed as separate 32-bit components. This representation is not ideal for the target accelerator because a single complex value occupies multiple architectural registers and requires several scalar instructions for each arithmetic operation.

Our first optimization is to change the accelerated FFT path to a packed fixed-point Q15 representation:

$$\texttt{word} = \{\texttt{imag}[15:0],\ \texttt{real}[15:0]\}.$$

One 32-bit integer register now contains one complete complex sample. This transformation is an acceleration mechanism in itself: it reduces register pressure, reduces memory traffic, and allows each CV-X-IF instruction to operate on a full complex value instead of only one scalar component.

This change also creates the arithmetic format expected by the coprocessor. The custom instructions operate directly on packed Q15 complex operands and implement the required fixed-point scaling and rounding inside the hardware datapath.

B. Packed Complex Custom Instructions

Once one register corresponds to one complex value, the most common FFT arithmetic macros can be replaced by packed complex custom instructions. These instructions process both real and imaginary components together and preserve the fixed-point rounding points required by the optimized implementation.

The first hardware version implements packed complex addition, subtraction, and multiplication with divide-by-4 scaling. This already removes a large number of scalar operations from the CV32A6 datapath while keeping the butterfly scheduling in software. (see fig 4)

C. Butterfly-Level Specialization

The final accelerator goes beyond generic complex arithmetic. Since FFT-512 is dominated by radix-4 butterflies, the most profitable operation is a structured butterfly kernel. The radix-2 stage is also accelerated, but with a smaller fused instruction sequence adapted to the tail of the transform.

For radix-2, the software uses `C_MULDIV2`, `BFLY2_ADD`, and `BFLY2_SUB`. These instructions combine fixed-point division, twiddle multiplication, and butterfly recombination.

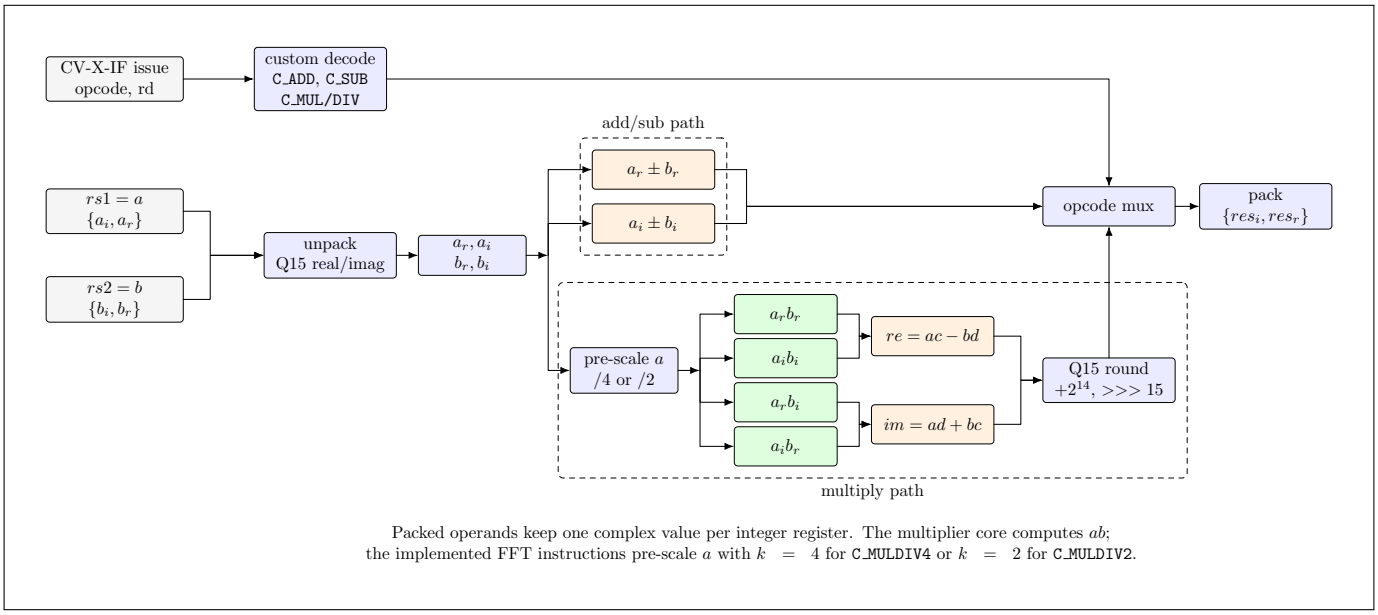

Fig. 4: Packed complex arithmetic datapath implemented in the CV-X-IF coprocessor.

For radix-4, a full butterfly requires four complex inputs and three twiddle factors. A single R-type instruction cannot provide all these operands. The coprocessor therefore uses a short stateful protocol: the software first loads the four inputs, then loads the first two twiddle factors, then executes the butterfly with the third twiddle factor, and finally reads back the buffered outputs.

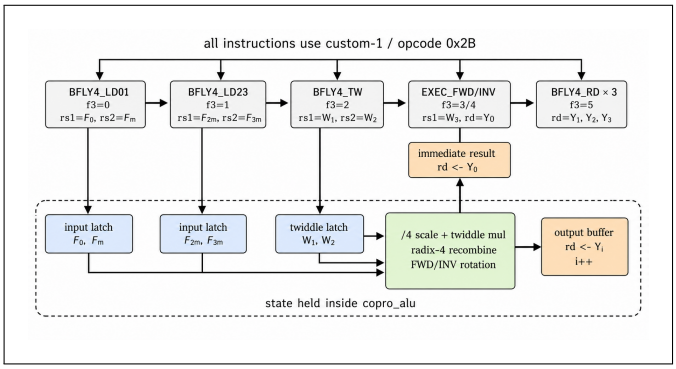

Fig. 5: Stateful radix-4 butterfly protocol implemented as a CV-X-IF custom-instruction sequence.

This design is intentionally more limited than a standalone FFT accelerator. The CV32A6 processor still controls loop structure, memory addressing, and stage traversal. The coprocessor only accelerates one butterfly instance at a time.

D. Implemented Custom Instruction Set

Listing 2 summarizes the software-visible custom instructions implemented in the FFT library and decoded by the coprocessor. All operands are packed Q15 complex values stored as `{imag[15:0], real[15:0]}` in one 32-bit integer register.

IV. Software Mapping

The software strategy is deliberately conservative. The KISS FFT decomposition, recursion, and twiddle tables are kept intact. Only the data representation, arithmetic macros, and hot butterfly kernels are retargeted to inline custom instructions. This makes validation easier because the optimized implementation remains close to the reference.

The first software modification converts FFT data from the original scalar representation to the packed Q15 representation expected by the coprocessor. After this conversion, each complex sample can be passed through a single 32-bit integer register

In the first hardware optimization step, the FFT library macros are redirected to `C_ADD`, `C_SUB`, and `C_MULDIV4`. The butterfly ordering remains software-controlled.

In the final step, the radix-2 and radix-4 kernels are rewritten more explicitly. The radix-2 tail uses the fused `BFLY2_*` operations. The radix-4 kernel uses the `BFLY4_*` protocol, so one software butterfly instance becomes a short sequence of custom instructions rather than a long sequence of scalar operations.

V. Results

This section presents the correctness, performance, timing, and resource measurements of the proposed solution. The results include both Questa simulation and execution on the Zybo FPGA board.

Listing 2: Custom FFT instructions implemented through CV-X-IF All operands are packed complex values.

```
Step 1: Packed complex arithmetic
-------------------------------------------------------------------------------------------------
Instruction       Encoding           Operands              Function
-------------------------------------------------------------------------------------------------
C_ADD             custom-3 / f3=1    rd, rs1=a, rs2=b      rd = a + b
C_SUB             custom-3 / f3=2    rd, rs1=a, rs2=b      rd = a - b
C_MULDIV4         custom-3 / f3=3    rd, rs1=a, rs2=b      rd = (a / 4) * b, Q15 rounded

Step 2: Fused fixed-point operators and radix-2 kernels
-------------------------------------------------------------------------------------------------
Instruction       Encoding           Operands              Function
-------------------------------------------------------------------------------------------------
C_ADD_ROT         custom-3 / f3=4    rd, rs1=a, rs2=b      rd = a + j*b
C_SUB_ROT         custom-3 / f3=5    rd, rs1=a, rs2=b      rd = a - j*b
C_FIXDIV4         custom-3 / f3=7    rd, rs1=a, rs2=dummy  rd = a / 4
C_MULDIV2         custom-2 / f3=3    rd, rs1=a, rs2=b      rd = (a / 2) * b, Q15 rounded
BFLY2_ADD         custom-2 / f3=1    rd, rs1=a, rs2=b      rd = (a / 2) + b
BFLY2_SUB         custom-2 / f3=2    rd, rs1=a, rs2=b      rd = (a / 2) - b

Step 3: Stateful radix-4 butterfly engine
-------------------------------------------------------------------------------------------------
Instruction       Encoding           Operands              Function
-------------------------------------------------------------------------------------------------
BFLY4_LD01        custom-1 / f3=0    rs1=f0,  rs2=fm       latch inputs f0 and fm
BFLY4_LD23        custom-1 / f3=1    rs1=fm2, rs2=fm3      latch inputs fm2 and fm3
BFLY4_TW          custom-1 / f3=2    rs1=tw1, rs2=tw2      latch twiddle factors tw1 and tw2
BFLY4_EXEC_FWD    custom-1 / f3=3    rd, rs1=tw3           compute forward radix-4; return out0
BFLY4_EXEC_INV    custom-1 / f3=4    rd, rs1=tw3           compute inverse radix-4; return out0
BFLY4_RD          custom-1 / f3=5    rd                    return next buffered output
```

A. Functional Validation

The modified processor must accelerate the FFT workload without breaking the rest of the contest software. Table I summarizes the functional validation. The FFT output passes in both simulation and FPGA execution. CoreMark and MNIST also execute correctly, which confirms that the custom coprocessor does not introduce a regression in unrelated programs (see proof in fig. 6,7).

TABLE I: Functional Validation Summary

Application	Simulation	Zybo board	Evidence
FFT	PASS	PASS	44811 instr.; 64980/75431 cyc.
CoreMark	2.624513	2.519275	Valid CoreMark 1.0 score
MNIST	1/1, 82	1/1, 82	Pred.=Exp.=4

B. FFT Performance

Table II reports the FFT instruction and cycle counts. In simulation, the optimized implementation reduces the instruction count by 63.9% and the cycle count by 57.7%. On the FPGA board, the instruction count reduction is the same and the cycle count is reduced by 55.4%.

C. Timing and FPGA Resource Usage

The timing report of the optimized design gives a worst setup slack of -5.089 ns under a 25 ns clock constraint.

TABLE II: FFT Performance

Configuration	Instructions	Cycles	Instr. change	Cycle change
Baseline simulation	124142	153496	–	–
Optimized simulation	44811	64980	-63.9%	-57.7%
Baseline Zybo board	124142	169248	–	–
Optimized Zybo board	44811	75431	-63.9%	-55.4%

This corresponds to an estimated maximum frequency of approximately

$$f_{\max} = \frac{1}{25.000 + 5.089} \simeq 33.24 \text{ MHz}.$$

Compared with the 40 MHz reference target, this represents a frequency degradation of approximately 17%. The optimized design is therefore within the authorized 20% decrease limit, since a 20% decrease from 40 MHz gives a minimum allowed frequency of 32 MHz.

Although the optimized design has negative slack at the original 25 ns target period, its estimated maximum frequency remains above the 32 MHz limit. The timing degradation is caused by the longer arithmetic path introduced by the coprocessor, especially in the radix-4 butterfly datapath. The design therefore trades part of the available

clock-frequency margin for a lower FFT cycle count while still satisfying the contest timing constraint.

TABLE III: Timing Report

Configuration	Constraint (ns)	Slack (ns)	f_{max} (MHz)	Change
Baseline	25.000	0.094	40.15	–
Optimized	25.000	-5.089	33.24	-17.2%

Table IV reports top-level FPGA resource usage. The optimized design uses more DSP blocks because the coprocessor implements dedicated fixed-point multiplication paths. LUT and flip-flop usage are slightly lower at top level in the reported build, while BRAM usage remains unchanged.

TABLE IV: Top-Level FPGA Resource Usage

Configuration	LUTs	FFs	RAM36	RAM18	DSPs
Baseline	21996	16438	60	5	4
Optimized	21427	15002	60	5	29
Change (%)	-2.6%	-8.7%	0.0%	0.0%	+625.0%

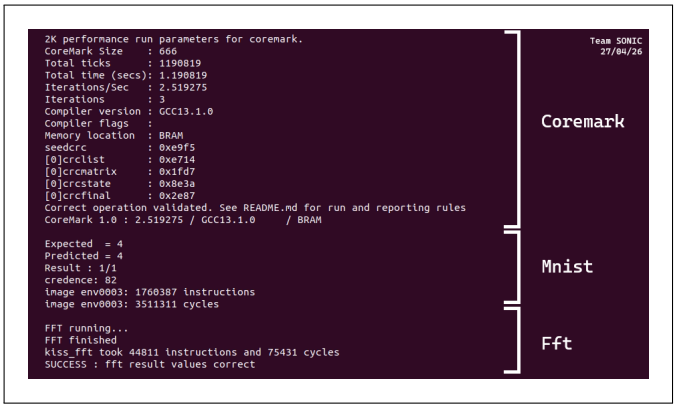

Fig. 6: Proof of Zybo board functional validation.

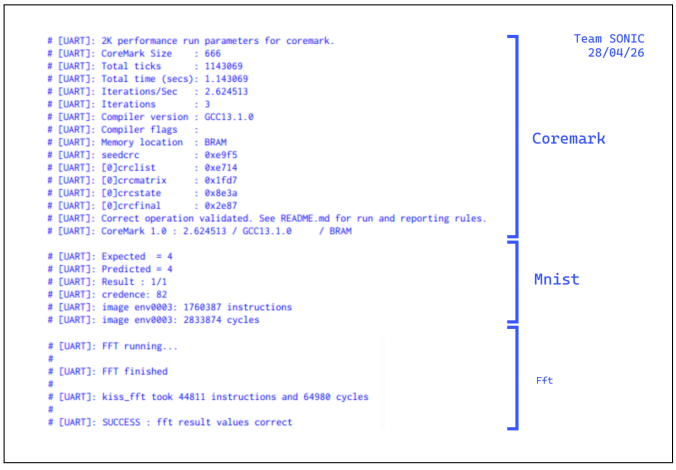

Fig. 7: Proof of Questa simulation functional validation.

VI. Discussion

The results confirm that the most useful optimization is not only replacing scalar multiplication with hardware multiplication. The main benefit comes from adapting the granularity of the software and hardware interface to the structure of the FFT. Converting the accelerated path to packed Q15 reduces data movement overhead. Packed complex instructions remove repeated scalar sequences. Finally, the radix-4 protocol removes a large part of the butterfly recombination overhead in the dominant FFT stages.

The main limitation of the final design is timing. The optimized implementation reduces cycle count substantially, but the post-route critical path becomes longer. Future work should therefore focus on pipelining the Q15 complex multiplier and the radix-4 recombination path, or on splitting the butterfly execution over more CV-X-IF cycles to recover timing while preserving most of the instruction-count reduction.

VII. Conclusion

This paper presented a CV-X-IF coprocessor for accelerating an FFT-512 workload on the CV32A6 RISC-V processor. The baseline software manipulates complex values as separate 32-bit real and imaginary components. The proposed solution first transforms the accelerated FFT path to a packed Q15 format so that one 32-bit register stores one complete complex value. It then adds packed complex arithmetic instructions and stage-aware butterfly instructions for the radix-2 and radix-4 FFT kernels.

The final optimized implementation reduces the FFT instruction count from 124142 to 44811 instructions. On the Zybo board, the FFT cycle count is reduced from 169248 to 75431 cycles (153496 $\rightarrow$ 64980 cycles for simulation), while FFT, CoreMark, and MNIST remain functionally valid. The cost of this specialization is a larger DSP usage and a reduced post-route maximum frequency. The solution therefore demonstrates a clear architectural trade-off: moving from scalar 32-bit complex arithmetic to packed fixed-point and butterfly-level instructions significantly reduces execution cycles, but the arithmetic path must be carefully pipelined to meet FPGA timing constraints.

Acknowledgments

We would like to thank the professors and supervisors at IMT Atlantique who supported this project and the organizers of the RISC-V student contest for providing the hardware and software framework used in this work.

References

[1] Thales Group, "CVA6 softcore contest repository," GitHub repository. [Online]. Available: https://github.com/ThalesGroup/cva6-softcore-contest

[2] M. Borgerding, "KISS FFT," GitHub repository. [Online]. Available: https://github.com/mborgerding/kissfft

[3] Sonic, "Public GitHub repository" [Online]. Available: https://github.com/sonic-cva6-team/cva6-softcore-contest-sonic